

Arrangement Coach Extension (Ableton Extensions SDK)

Product + Engineering Roadmap

1. Overview

The Arrangement Coach is an Ableton Live Extension that provides **non-destructive, intelligent arrangement guidance** for producers.

It does NOT generate music or move clips.

Instead, it acts as:

- A structural analyzer
- A transition advisor
- A production checklist system
- A real-time arrangement “coach”

It uses:

- **Locators = structural anchors**
 - **Extension UI = notes + intelligence layer**
 - Optional metadata mapping between locators and section data
-

2. Core Design Principles

2.1 Non-destructive

Never modify audio or MIDI unless explicitly triggered by the user.

2.2 Producer-in-control

The extension suggests, not performs arrangement changes.

2.3 Structure-first

Locators define the song architecture.

2.4 UI-first intelligence

All “notes”, suggestions, and analysis live in the extension UI.

2.5 Lightweight integration

Avoid deep modification of Ableton internals.

3. System Architecture

3.1 Data Model

Section (core object)

Represents a locator-defined region.

```
Section {  
  id: string  
  name: string  
  startBar: number  
  endBar: number  
  
  notes: Note[]  
  issues: Issue[]  
  transitions: TransitionSuggestion[]  
  checklist: ChecklistItem[]  
  
  energyScore?: number  
  trackDensity?: number  
}
```

Note

```
Note {  
  id: string  
  text: string  
  severity: "info" | "warning" | "critical"  
}
```

Transition Suggestion

```
TransitionSuggestion {  
  fromSection: string  
  toSection: string  
  
  type: "riser" | "impact" | "drum_fill" | "filter" | "reverse" | "silence"  
  durationBars: number  
  notes: string[]  
}
```

Checklist Item

```
ChecklistItem {  
  id: string  
  text: string  
  completed: boolean  
}
```

4. Ableton Integration Layer

4.1 Locators (Primary Mapping System)

- Each locator = Section boundary
- Locator name = Section name
- Locator position = start bar/time

Mapping rule:

```
Locator[i] → Section[i]
```

4.2 Playhead Tracking

- Detect current playhead position
- Resolve active section
- Update UI context

4.3 Arrangement Reading

Collect:

- Track count
- Clip density per section
- MIDI note density
- Automation presence (if available)

Used for:

- energy estimation
 - issue detection
 - transition suggestions
-

5. Extension UI Design

5.1 Main Panel Layout

Header

- Current Section Name
 - Energy Score (optional)
 - Time range (bars)
-

Section Notes Panel

Displays:

- AI-generated notes
 - user-added notes
 - warnings
-

Transition Panel

Displays:

- Suggested transition into next section
 - checklist of required elements
-

Checklist Panel

Per-section production tasks

Issues Panel

Detects structural problems:

- flat energy
 - missing transition
 - over-repetition
 - imbalance
-

6. Core Features

6.1 Arrangement Map Builder

On load:

- Scan locators
 - Build Section objects
 - Initialize mapping
-

6.2 Section Analyzer

For each section:

- calculate track density
- detect repetition vs previous section
- evaluate energy delta
- detect missing transitions

Outputs:

- notes
 - issues
 - checklist items
-

6.3 Transition Engine

For each boundary:

Section A → Section B

Generate:

- transition type recommendation
- duration suggestion
- checklist items

Examples:

- snare roll
 - riser
 - impact hit
 - filter sweep
 - drum dropout
-

6.4 Genre Rules Engine

Apply structural expectations:

Example: Melodic Techno

- long builds (16–32 bars)
- gradual energy transitions
- restrained drops

Example: Uplifting Trance

- breakdown emphasis
- layered supersaws in drop
- long risers

Output:

- warnings
 - suggestions
 - structural alignment score
-

6.5 Energy Model (Heuristic-based)

Compute per section:

- number of active tracks
- density of MIDI notes
- automation presence
- instrument layering

Output:

Energy Score: 1-10

Used for:

- detecting flat sections
 - identifying weak transitions
-

6.6 Issue Detection Engine

Detect:

- Flat energy sections
 - Missing transition elements
 - Repeated drop without variation
 - Abrupt energy changes
 - Overcrowded frequency regions (heuristic)
-

7. UI State Management

Global State

```
ProjectState {  
  sections: Section[]  
  activeSectionId: string  
  genre: string  
  energyCurve: number[]  
}
```

Sync rules

- Locator click → update activeSection
 - Playhead move → update activeSection
 - Section selection → refresh UI panels
-

8. MVP Scope

Phase 1 (Core)

- Locator scanning
 - Section model creation
 - UI panel showing section notes
 - Manual notes per section
 - Basic transition suggestions
-

Phase 2 (Analysis)

- Energy estimation
 - Issue detection
 - Checklist system
-

Phase 3 (Intelligence Layer)

- Genre rules engine
 - Transition optimization
 - Structural scoring
-

Phase 4 (Polish)

- Visual energy graph
 - Keyboard navigation between sections
 - Quick-add note system
 - "Fix this section" button
-

9. Non-Goals

Do NOT implement:

- Audio generation
 - MIDI generation
 - Automatic arrangement rewriting
 - AI composition replacement
-

10. Success Criteria

The extension is successful if:

- A producer can understand their arrangement in <10 seconds
 - Weak transitions are consistently identified
 - Users rely on suggestions during real production
 - No disruption to Ableton workflow
 - Works as a “second brain” for arrangement decisions
-

11. Future Expansion Ideas

- Reference track alignment
 - Automated arrangement comparisons
 - Collaborative notes layer
 - Exportable arrangement report (PDF / markdown)
 - Integration with sample libraries for transition suggestions
 - “Arrangement scoring” leaderboard per project
-